

AI Studio Floor Plan Solver — Master Development Plan

AI STUDIO FLOOR PLAN SOLVER — MASTER DEVELOPMENT PLAN v1.0

Project: Genuine Stuffs AI Studio — Floor Plan Generation Rebuild **Stack:** TypeScript, Next.js, deployed on Vercel (genuinestuffs.com) **Program generator:** Hive (Gemini 2.5 Flash via OpenRouter) — already live, DO NOT MODIFY **Placement engine:** Grid-discretized constraint solver (replaces squarify) **Audience of this document:** Any AI model or developer executing a phase. Follow it literally. Where this document and your own judgment conflict, this document wins. If this document is genuinely ambiguous, STOP and ask the human — do not guess.

PART A — NON-NEGOTIABLE RULES (READ BEFORE EVERY SESSION)

A1. Locked decisions (do not re-litigate, do not “improve”)

#	Decision	Value
D1	Placement algorithm	Custom TypeScript backtracking constraint solver. NO external CP library, NO WASM, NO Python microservice, NO ML model.
D2	Room geometry (v1)	Axis-aligned rectangles on a grid. Non-rectangular (L-shaped) rooms are OUT OF SCOPE for v1.
D3	Grid resolution	0.5 m per cell, configurable via one constant <code>GRID_RESOLUTION_M = 0.5</code> . Never hardcode 0.5 anywhere else.
D4	Solver time budget	Default 8000 ms, configurable via <code>SOLVER_BUDGET_MS</code> . Solver MUST return within budget with best-effort status.
D5	Door adjacency rule	Two rooms are “adjacent” only if they share a wall segment ≥ 2 contiguous cells (= 1.0 m at 0.5 m grid).
D6	Hive contract	The Hive’s JSON output format is frozen. The normalizer adapts to the Hive; the Hive is never changed to suit the solver.
D7	squarify	Remains in the codebase untouched until Phase 6 sign-off, behind the legacy code path. Deleted only in Phase 7.
D8	Failure mode	The solver NEVER returns invalid geometry. It returns a typed failure (<code>UNSAT / TIMEOUT</code>) with diagnostics instead.

A2. Session protocol (every AI session, every time)

- One phase per session.** Never start a phase before the previous phase’s Definition of Done is checked off by the human.
- Request current file state** before writing any diff. Never edit a file from memory of a previous session.

3. **Surgical edits only.** Touch only files listed in the current phase's "Files touched" table. If you believe an out-of-scope change is necessary, state the file, the reason, and WAIT for approval before writing it.
4. **Run the harness first and last.** Every session begins and ends with `npm run harness`. If it fails at session start, fix nothing else until the human confirms whether the failure is expected.
5. **No silent renames, no drive-by refactors, no formatting sweeps, no dependency additions** without explicit approval.
6. **Commit message format:** phase-N: <what changed> [harness: X/Y passing].

A3. Coding standards

- Strict TypeScript. "strict": true. No any. No // @ts-ignore.
- Every exported function has a JSDoc block stating: purpose, inputs, outputs, failure behavior.
- All units are METERS in the domain layer and CELLS in the solver layer. Every variable carrying a length ends in `_m` or `_cells`. Conversion happens in exactly one file: `units.ts`.
- No function longer than 60 lines. Split before you exceed it.
- Pure functions everywhere except the entry points. No module-level mutable state.
- Every error thrown or returned carries an error code from the taxonomy in Part C.

PART B — DATA CONTRACTS (THE SPINE OF THE SYSTEM)

All types live in `src/floorplan/types.ts`. This file is written in Phase 1 and is APPEND-ONLY afterward: later phases may add types but never change or remove existing fields. Any field change requires human approval and a version bump of this document.

```
// ----- 1. RAW HIVE OUTPUT (as received – trust nothing) -----
/** Exactly mirrors Hive JSON. All fields optional because LLM output is unreliable. */
export interface RawHiveRoom {
  id?: string;
  type?: string; // e.g. "bedroom", "kitchen", "living", "toilet", "corridor"
  label?: string;
  area_m2?: number;
  min_width_m?: number;
  adjacent_to?: string[]; // room ids
  needs_window?: boolean;
  notes?: string;
}

export interface RawHiveProgram {
  rooms?: RawHiveRoom[];
  footprint?: { width_m?: number; length_m?: number };
  meta?: Record<string, unknown>;
}

// ----- 2. NORMALIZED PROGRAM (post-Phase-1 – fully trusted) -----
/** Every field REQUIRED and validated. Downstream code never re-checks these invariants. */
export interface NormRoom {
  id: string; // unique, non-empty, kebab-case
  type: RoomType; // closed enum, see below
  targetArea_m2: number; // > 0
  minWidth_m: number; // >= ABSOLUTE_MIN_WIDTH_M for its type
  adjacentTo: string[]; // only ids that exist; deduplicated; no self-reference; symmetric
  needsWindow: boolean; // resolved from type defaults if Hive omitted it
  isHubEligible: boolean; // computed by graph module (Phase 2)
}

export type RoomType =
  | "living" | "bedroom" | "kitchen" | "toilet" | "bathroom"
  | "corridor" | "dining" | "store" | "garage" | "balcony" | "other";

export interface NormProgram {
  rooms: NormRoom[]; // sorted: hub first, then area DESC
  footprint: { width_m: number; length_m: number };
  gridResolution_m: number;
  warnings: NormWarning[]; // non-fatal issues, surfaced to UI
}

export interface NormWarning { code: string; roomId?: string; message: string; }
```

```
// ----- 3. FEASIBILITY REPORT (gate between Phase 1 and Phase 3) -----
export interface FeasibilityReport {
  feasible: boolean;
  checks: FeasibilityCheck[];
}
export interface FeasibilityCheck {
  code: string; // from error taxonomy, F-* series
  passed: boolean;
  detail: string; // human-readable, includes actual numbers
}

// ----- 4. SOLVER I/O -----
export interface SolverConfig {
  budget_ms: number; // default 8000
  areaTolerance: number; // default 0.10 (±10%), relaxable to 0.20
  seed: number; // for reproducible variant generation
}

export interface PlacedRoom {
  roomId: string;
  x_cells: number; y_cells: number; // top-left, 0-indexed
  w_cells: number; h_cells: number; // both >= minWidth in cells
}

export type SolveStatus = "SOLVED" | "SOLVED_RELAXED" | "UNSAT" | "TIMEOUT";

export interface SolveResult {
  status: SolveStatus;
  placements: PlacedRoom[]; // EMPTY unless status starts with "SOLVED"
  relaxationsApplied: string[]; // codes from relaxation ladder, in order applied
  diagnostics: SolveDiagnostics;
}

export interface SolveDiagnostics {
  elapsed_ms: number;
  nodesExplored: number;
  failedConstraint?: string; // deepest constraint that blocked search, for UNSAT debugging
}

// ----- 5. FINAL GEOMETRY (post-refinement, consumed by renderer) -----
export interface FloorPlan {
  rooms: Array<{ roomId: string; x_m: number; y_m: number; w_m: number; h_m: number }>;
  doors: Array<{ roomA: string; roomB: string; wallMidpoint_m: { x: number; y: number } }>;
  score: number; // 0..100, from Phase 5 scorer
  provenance: { solverSeed: number; relaxationsApplied: string[] };
}
```

Invariant table (memorize; every phase’s tests assert these):

ID	Invariant	Enforced by
I1	NormProgram adjacency is symmetric and references only existing room ids	Phase 1 normalizer
I2	$\Sigma \text{targetArea} \leq 0.85 \times \text{footprint area}$ (15% reserved for walls/circulation)	Phase 1 feasibility gate F-001
I3	Placements never overlap and never exceed the footprint	Phase 3 solver, by construction
I4	Every declared adjacency in a SOLVED result shares ≥ 2 contiguous edge cells	Phase 3 solver, by construction
I5	Every needsWindow room touches the footprint perimeter in a SOLVED result	Phase 3 solver, by construction
I6	SolveResult.placements is empty unless status is SOLVED or SOLVED_RELAXED	Phase 3 solver return path
I7	Refinement (Phase 4) never changes topology — only wall coordinates	Phase 4 post-check re-runs I3-I5

PART C — ERROR TAXONOMY (COMPLETE; DO NOT INVENT

NEW CODES WITHOUT APPROVAL)

Code	Layer	Meaning	Action
N-001	Normalizer	Room missing/empty id	Auto-generate room- {index}-{type}; add warning
N-002	Normalizer	Unknown room type string	Map to "other"; add warning
N-003	Normalizer	Non-positive or absurd area (≤ 0 or $> 200 \text{ m}^2$)	Clamp to type default table; add warning
N-004	Normalizer	Adjacency references unknown id	Drop that edge; add warning
N-005	Normalizer	Asymmetric adjacency (A lists B, B omits A)	Make symmetric (add missing side); add warning
N-006	Normalizer	Duplicate room ids	Suffix -2, -3...; add warning
N-007	Normalizer	Hive JSON unparseable / rooms array missing	FATAL — return typed error, never a partial program
F-001	Feasibility	Area budget exceeded (violates I2)	Feasible=false; detail shows Σarea vs $0.85 \times \text{footprint}$
F-002	Feasibility	Some room's minWidth > footprint's shorter side	Feasible=false; name the room
F-003	Feasibility	Adjacency graph disconnected (unreachable rooms)	Feasible=false; list unreachable ids
F-004	Feasibility	A room's adjacency degree > 6	Feasible=false (geometrically implausible for rectangles)
F-005	Feasibility	More needsWindow rooms than perimeter can host (Σ their minWidths > perimeter length $\times 0.9$)	Feasible=false
S-001	Solver	UNSAT after full relaxation ladder	Return UNSAT + deepest failed constraint
S-002	Solver	Time budget exhausted	Return TIMEOUT + best diagnostics
R-001	Refiner	Refined geometry failed I3-I5 re-check	FATAL — discard refinement, return the raw solved grid geometry (which is valid)

Relaxation ladder (Phase 3, applied strictly in this order, one rung at a time, re-solving after each):

1. RELAX-AREA-20: widen area tolerance from $\pm 10\%$ to $\pm 20\%$.
2. RELAX-SOFT-ADJ: drop adjacencies where neither room is the hub and neither is a corridor (record which were dropped).
3. RELAX-MINWIDTH: reduce all minWidths to the ABSOLUTE floor table (toilet 1.2 m, bathroom 1.5 m, corridor 1.0 m, all others 2.4 m).
4. Nothing left $\rightarrow$ return UNSAT with S-001.

PART D — PHASE PLAN

Phases execute strictly in order. Each phase lists: objective, files touched, step-by-step tasks, forbidden actions, and a Definition of Done (DoD) the human checks off before the next phase begins.

PHASE 0 — Fixture & Regression Harness (build the test bed BEFORE any engine code)

Objective: A frozen set of real Hive outputs plus a one-command harness, so every later phase is measured, not vibed.

Files touched (create only):

```
src/floorplan/fixtures/hive-001-3bed-flat.json      ... hive-012-*.json
src/floorplan/fixtures/hive-101-overconstrained.json (adjacency that cannot planarize)
src/floorplan/fixtures/hive-102-overbudget.json     ( $\Sigma \text{area} > 0.85 \times \text{footprint}$ )
src/floorplan/fixtures/hive-103-garbage.json        (missing ids, wrong types, asymmetric adjacency)
src/floorplan/harness/run.ts
src/floorplan/harness/assertions.ts
package.json                                         (add script "harness" only – no dependency changes)
```

Tasks, in order:

1. Capture 12 real Hive responses from production logs verbatim into hive-001 ... hive-012. Do not clean them up — the mess IS the test.
2. Hand-author the 3 nasty fixtures (101-103) exactly as described above.
3. Write assertions.ts exporting pure predicate functions, one per invariant I1-I7, each returning { pass: boolean; detail: string }.
4. Write run.ts: loads every fixture, runs whatever pipeline stages currently exist (in Phase 0, that is none — it only validates that fixtures parse as JSON), prints a table: fixture | stage reached | pass/fail | detail. Exit code 1 on any failure.
5. Add "harness": "tsx src/floorplan/harness/run.ts" to package.json scripts.

Forbidden: touching any existing solver/squarify/render code; adding npm dependencies other than tsx as a devDependency if not already present (ask first).

DoD: npm run harness runs green on all 15 fixtures (Phase 0 pass = “parses as JSON”); fixtures committed; human has eyeballed fixture list.

PHASE 1 — Ingestion Normalizer + Feasibility Gate

Objective: normalizeHiveProgram() turns any RawHiveProgram into either a fully-trusted NormProgram + FeasibilityReport, or a typed fatal error. After this phase, NO downstream code ever handles dirty data.

Files touched:

```
src/floorplan/types.ts      (create – full contract from Part B)
src/floorplan/units.ts      (create – metersToCells / cellsToMeters, the ONLY conversion site)
src/floorplan/normalize.ts  (create)
src/floorplan/feasibility.ts (create)
src/floorplan/defaults.ts   (create – type default tables below)
src/floorplan/harness/run.ts (extend: pipe fixtures through normalize + feasibility)
```

Default tables (put in defaults.ts, values are FINAL unless human amends):

```
export const ABSOLUTE_MIN_WIDTH_M: Record<RoomType, number> = {
  living: 3.0, bedroom: 2.7, kitchen: 2.1, toilet: 1.2, bathroom: 1.5,
  corridor: 1.0, dining: 2.7, store: 1.2, garage: 3.0, balcony: 1.2, other: 2.4,
};
export const DEFAULT_AREA_M2: Record<RoomType, number> = {
  living: 24, bedroom: 12, kitchen: 9, toilet: 2.5, bathroom: 4,
  corridor: 6, dining: 12, store: 3, garage: 18, balcony: 4, other: 9,
};
export const NEEDS_WINDOW_DEFAULT: Record<RoomType, boolean> = {
  living: true, bedroom: true, kitchen: true, toilet: false, bathroom: false,
  corridor: false, dining: true, store: false, garage: false, balcony: true, other: false,
};
export const AREA_BUDGET_RATIO = 0.85; // invariant I2
export const MAX_ADJACENCY_DEGREE = 6; // check F-004
```

Tasks, in order:

1. Write types.ts verbatim from Part B. No edits, no “improvements”.

2. Write `units.ts: metersToCells(m: number): number = Math.round(m / GRID_RESOLUTION_M)`, floor variant for widths uses `Math.max(1, Math.floor(...))`. Export `GRID_RESOLUTION_M = 0.5`.
3. Write `normalize.ts` implementing rules N-001 ... N-007 EXACTLY as the taxonomy states. Signature: `normalizeHiveProgram(raw: unknown): { ok: true; program: NormProgram } | { ok: false; code: "N-007"; message: string }`. Parse defensively: raw is unknown, not `RawHiveProgram` — check every field's existence and type before use.
4. Sort rooms in the output: hub-eligible first (until Phase 2 lands, treat type === "living" as the provisional hub rule and mark it // PROVISIONAL – replaced in Phase 2), then remaining rooms by `targetArea` descending.
5. Write `feasibility.ts` implementing F-001 ... F-005 exactly. Signature: `checkFeasibility(p: NormProgram): FeasibilityReport`. Every check runs even after one fails (report is complete, not short-circuited).
6. Extend the harness: fixtures 001-012 and 103 must normalize with `ok: true` (103 with many warnings); 101 must fail feasibility on F-003 or F-004; 102 must fail on F-001. Encode these expectations as named assertions in the harness, not comments.

Forbidden: importing anything from solver/placement code (doesn't exist yet — keep it that way); network calls; touching the Hive prompt.

DoD: harness green with the exact expected outcomes above; every taxonomy code N-001...F-005 has at least one fixture that triggers it; human reviews the warnings printed for fixture 103.

PHASE 2 — Graph Module Fix (hub eligibility)

Objective: Correct hub selection in the existing `graph.ts` and wire it into normalization, replacing the provisional rule.

Files touched: `src/floorplan/graph.ts` (existing — request current contents first), `normalize.ts` (swap provisional hub rule), harness.

Hub-eligibility rule (FINAL): a room is hub-eligible iff (a) its type is `living` or `corridor`, AND (b) its adjacency degree is ≥ 2 , AND (c) it is not a leaf on the graph's spanning tree. The hub = the eligible room with the highest degree; ties broken by larger `targetArea`; if none eligible, the room with highest degree overall becomes hub with warning N-008: no natural hub (add N-008 to taxonomy — pre-approved).

Tasks: 1) Request current `graph.ts`. 2) Implement `computeHub(rooms: NormRoom[]): string` per the rule. 3) Implement `isConnected(rooms): boolean` and `unreachableFrom(hubId, rooms): string[]` for F-003 to consume. 4) Replace the provisional line in `normalize.ts`. 5) Add two fixtures: one corridor-hub plan, one no-natural-hub plan; assert hub choice by id in the harness.

Forbidden: changing `graph.ts` function signatures already consumed elsewhere without listing every call site first.

DoD: harness green; hub assertions pass on both new fixtures; fixture 101's F-003 detail now names the exact unreachable room ids.

PHASE 3 — Constraint Placement Engine (the core; budget two sessions)

Objective: `solve(program, config): SolveResult` — a backtracking solver placing rooms as rectangles on the grid, satisfying I3, I4, I5 BY CONSTRUCTION, honoring the relaxation ladder and the time budget.

Files touched (create only):

<code>src/floorplan/solver/grid.ts</code>	(occupancy grid: set/clear/query rectangles, edge-contact queries)
<code>src/floorplan/solver/candidates.ts</code>	(candidate rectangle enumeration per room)
<code>src/floorplan/solver/constraints.ts</code>	(pure predicate per constraint, each ≤ 25 lines)
<code>src/floorplan/solver/search.ts</code>	(backtracking loop + ordering heuristics + budget clock)
<code>src/floorplan/solver/relax.ts</code>	(relaxation ladder driver)
<code>src/floorplan/solver/index.ts</code>	(public <code>solve()</code> — the ONLY export other layers may import)

Session 3a — grid + candidates + constraints (no search yet):

1. `grid.ts: class OccupancyGrid` backed by `Uint16Array` (0 = empty, else `roomIndex+1`). Methods: `canPlace(rect): boolean`, `place(rect, roomIdx): void`, `remove(rect): void`, `sharedEdgeCells(rectA, rectB): number`, `touchesPerimeter(rect): boolean`. Every method O(rect area) or better; no allocations inside

loops.

2. `candidates.ts`: for a room, enumerate candidate rects: for each (`w_cells`, `h_cells`) pair where `w*h*cellArea` is within `areaTolerance` of `targetArea` AND `min(w,h) >= minWidth_cells` AND aspect ratio $\leq 3:1$ — for each valid (`x`, `y`) on the grid. Return as a generator (lazy), ordered by the position heuristics below. Cap enumerated dimension pairs at 12 per room (closest to target area first).
3. `constraints.ts`: pure functions — `noOverlap`, `insideFootprint`, `adjacencySatisfied`(`placed`, `room`, `rect`) (checks D5's ≥ 2 shared edge cells against every ALREADY-PLACED declared neighbor), `windowSatisfied`. Each returns { `pass`: boolean; `code`: string }.
4. Unit-test each in the harness against hand-built 10×10 grids (add `harness/solver-units.ts`; wire into `npm run harness`).

Session 3b — search + relaxation + budget:

5. `search.ts`: depth-first backtracking. **Room order**: exactly the NormProgram order (hub first, then area desc) — do not re-sort. **Position ordering heuristic**: hub candidates ordered by distance from footprint center ASC; `needsWindow` rooms by perimeter-contact length DESC; all others by (shared-edge cells with already-placed declared neighbors) DESC then distance-to-hub ASC. **Pruning**: before recursing, verify every UNPLACED room still has ≥ 1 feasible candidate given remaining free cells (cheap check: remaining free area $\geq \Sigma$ remaining min areas; if not, backtrack). **Budget**: check `performance.now()` every 200 nodes; on expiry return best partial diagnostics with status TIMEOUT.
6. `relax.ts`: run solve at rung 0; on UNSAT apply rung 1, re-solve; continue per Part C ladder. Record every rung applied in `relaxationsApplied`. Status is SOLVED_RELAXED if any rung > 0 succeeded.
7. `index.ts`: export function `solve(program: NormProgram, config: SolverConfig): SolveResult`. Deterministic for a given seed: the seed permutes candidate order via a seeded PRNG (implement `xorshift32` in `search.ts`, 10 lines, no dependency).
8. Extend harness: fixtures 001-012 must reach SOLVED or SOLVED_RELAXED within budget; assert I3, I4, I5 on every result via `assertions.ts`; fixture 101 must return UNSAT with a named `failedConstraint`; record `elapsed_ms` per fixture in the harness table.

Forbidden: any post-hoc “repair” of an invalid layout (that is the old squarify mindset — if a rect violates a constraint it is never placed); recursion without the budget clock; touching `normalize/feasibility/graph` files.

DoD: harness green; all 12 real fixtures solve in < 8000 ms each (< 2000 ms median); zero invariant assertion failures across 5 different seeds (run harness with seeds 1-5); human reviews the printed timing table.

PHASE 4 — Post-Solve Refinement (cosmetic only)

Objective: Snap the solved grid geometry to clean architectural dimensions WITHOUT changing topology (invariant I7).

Files touched (create): `src/floorplan/refine.ts`; harness.

Tasks: 1) Convert placements to meters via `units.ts`. 2) Snap every wall coordinate to the nearest 0.05 m. 3) Merge collinear wall segments. 4) Compute door positions: midpoint of the longest shared wall segment per declared adjacency (populates `FloorPlan.doors`). 5) **Mandatory post-check**: re-run I3/I4/I5 predicates on the refined geometry; on any failure, log R-001 and return the unrefined (still valid) geometry. Refinement failing is acceptable; refinement corrupting is not. 6) Harness: assert refined output passes I3-I5 for all solved fixtures and that every declared adjacency has exactly one door.

Forbidden: moving rooms, resizing beyond snap distance (> 0.1 m total per wall), “fixing” solver output.

DoD: harness green; visual spot-check by human of 3 rendered fixtures.

PHASE 5 — Multi-Variant Generation & Scoring

Objective: Generate 3-5 layout variants and rank them, so the UI shows options, not one take-it-or-leave-it plan.

Files touched (create): src/floorplan/score.ts, src/floorplan/variants.ts; harness.

Scoring rubric (FINAL weights; each sub-score normalized 0-100):

Weight	Criterion	Computation
0.30	Area fidelity	$100 - \text{mean}(\text{satisfied declared adjacencies} \div \text{total declared}) \times 100$; SOLVED_RELAXED plans count dropped edges as unsatisfied
0.25	Adjacency quality	mean room aspect-ratio score: 100 at 1:1 linearly down to 0 at 3:1
0.20	Compactness	mean perimeter-contact length $\div$ room width for needsWindow rooms, $\times 100$ capped
0.15	Window quality	hub's shared-edge total with neighbors $\div$ hub perimeter, $\times 100$ capped
0.10	Circulation	

Tasks: 1) score.ts implements the rubric exactly; returns the composite plus the 5 sub-scores (UI will show the breakdown). 2) variants.ts: run solve() with seeds [base, base+1, ... base+4], drop failures, refine + score survivors, deduplicate (two variants are duplicates if every room's rect matches within 1 cell), sort by score DESC, return top 3. Run seeds SEQUENTIALLY (shared budget: SOLVER_BUDGET_MS total across all seeds, split evenly). 3) Harness: assert ≥ 2 distinct variants for at least 8 of the 12 real fixtures; assert scores are monotone with the sort.

DoD: harness green; human reviews score breakdowns for 3 fixtures and confirms rankings match architectural intuition.

PHASE 6 — Integration, Legacy Parity Gate & Rollout

Objective: Wire the pipeline into the AI Studio route behind a flag; prove superiority over squarify; ship.

Files touched: the API route that currently calls squarify (request current contents first), src/floorplan/pipeline.ts (create — the one function composing normalize $\rightarrow$ feasibility $\rightarrow$ variants), env/flag config, harness.

Pipeline contract: generateFloorPlans(rawHive: unknown): { ok: true; plans: FloorPlan[] } | { ok: false; code: string; userMessage: string }. userMessage is plain-language, e.g. F-001 $\rightarrow$ "The rooms you asked for need about 128 m² but the plot allows about 102 m². Reduce room sizes or count." Every F-* and S-* code gets a mapped userMessage in one lookup table — no free-text error strings in the route.

Tasks: 1) Build pipeline.ts. 2) Flag NEXT_PUBLIC_SOLVER_V2 — when false, old squarify path runs untouched. 3) **Parity gate:** run both paths over all 12 real fixtures; produce a side-by-side table (invariant pass-rate, score, timing). The new path must pass 100% of invariants where squarify's pass rate is whatever it is (expected: far lower). 4) Human flips the flag in staging, tests 10 live Hive generations end-to-end. 5) Flip in production. 6) Monitor for one week: log every UNSAT/TIMEOUT with its fixture-ready raw Hive JSON so failures become new fixtures.

DoD: parity table reviewed; production flag on; zero R-001 or invariant failures in week-one logs.

PHASE 7 — Cleanup

Delete squarify path and flag; promote any week-one failure logs into numbered fixtures; tag floorplan-v2.0.0. DoD: harness green; dead code gone; this document updated to v1.1 with any amendments made along the way.

PART E — WHAT "DONE" MEANS FOR THE WHOLE PROJECT

1. npm run harness exercises all 15+ fixtures through the full pipeline and is green on seeds 1-5.
2. No fixture ever renders overlapping rooms, out-of-bounds rooms, unreachable rooms, or

windowless bedrooms — because the solver cannot produce them (I3–I5) and refinement cannot corrupt them (I7).

3. Every failure a user can see maps to one taxonomy code and one plain-language message.
4. A brand-new AI session can pick up any phase using only: this document + the current repo + the phase's "request current file state" step. If that is ever untrue, the fix is to amend THIS DOCUMENT, not to improvise.

Genuine Stuffs — Build right. Build safely. Build with confidence.